LeaseGPT Tutorial - Part 2: Backend API & The RAG Pipeline

Welcome to Part 2 of the LeaseGPT tutorial! In Part 1, we set up our database and project structure. Now, we're going to build the brain and the nervous system of our application.

We will build the **Backend API** (how our frontend will talk to our system) and the **RAG Pipeline** (how our system actually reads and understands leases).

Chapter 4: Building the Backend API (FastAPI)

Let's start by defining some key concepts before we write code.

Beginner Concepts

- **What is FastAPI?** FastAPI is a modern web framework for Python. Think of a web framework as a toolkit that makes it easy to build websites or APIs. FastAPI is special because it is extremely fast, automatically generates documentation for you, and uses modern Python features like "async" (meaning it can handle multiple requests at the same time without waiting).
- **What is a REST API?** API stands for Application Programming Interface. A REST API is a standard way for two computers to talk to each other over the internet using HTTP (the same protocol your web browser uses). Our React frontend will send "requests" to our FastAPI backend, and our backend will send "responses" back.
- **What are API routes/endpoints?** A route (or endpoint) is a specific URL that triggers a specific function in your code. For example, sending a request to `http://localhost:8000/health` might trigger a function that checks if the server is running.
- **What is CORS?** CORS stands for Cross-Origin Resource Sharing. By default, web browsers block web pages from making requests to a different domain or port for security reasons. Since our frontend will run on port 3000 and our backend on port 8000, we need to configure CORS to explicitly tell the browser, "It's okay, let port 3000 talk to port 8000."
- **What is Dependency Injection?** Imagine building a house. Instead of every worker building their own hammer, you have a tool shed where workers request a hammer when they need one. Dependency injection in FastAPI works similarly. Instead of every route opening its own database connection, routes simply state "I need a database connection," and FastAPI provides (injects) it automatically.
- **What is Middleware?** Middleware is code that runs *between* receiving a request and processing it, or *between* processing a request and sending the response. Think of it like a security checkpoint at a building. Every visitor (request) must pass through the checkpoint (middleware) before they can visit an office (route).

1. `backend/app/main.py` — The Application Factory

This is the entry point of our backend. It sets up the FastAPI application, configures middleware, and registers our routes.

```
# backend/app/main.py

import logging
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.routes import health, upload, chat
from app.api.middleware import RequestLoggingMiddleware
from app.db.session import engine
from app.models.base import Base
from app.config import settings

# Import all models so SQLAlchemy knows about them when we call create_all.
# Without this import, Base.metadata has zero tables and nothing gets created.
import app.models.lease  # noqa: F401

logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    The lifespan context manager.
    Code before the 'yield' runs when the application STARTS UP.
    Code after the 'yield' runs when the application SHUTS DOWN.
    """
    # ── Startup ──────────────────────────────────────────────────────
    logger.info("Starting up LeaseGPT backend …")

    # Create all database tables that don't already exist.
    # 'begin()' opens a connection from the pool; 'run_sync()' lets us call
    # the synchronous create_all() method inside our async context.
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    logger.info("Database tables verified / created.")

    # ── Application runs ─────────────────────────────────────────────
    yield

    # ── Shutdown ─────────────────────────────────────────────────────
    # Dispose the connection pool so every open connection is properly closed.
    # Without this, you can leak database connections on restarts.
    await engine.dispose()
    logger.info("Shutting down LeaseGPT backend …  Connection pool closed.")

# Create the actual FastAPI application instance
app = FastAPI(
    title=settings.PROJECT_NAME,
    description="API for uploading leases and chatting with them via RAG",
    version="1.0.0",
    lifespan=lifespan
)

# ── Middleware ──────────────────────────────────────────────────────────
# Middleware is applied in reverse order of registration.
# RequestLoggingMiddleware runs outermost (first in, last out).
app.add_middleware(RequestLoggingMiddleware)
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,  # React dev server
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ── Routers ─────────────────────────────────────────────────────────────
app.include_router(health.router, prefix="/health", tags=["Health"])
app.include_router(upload.router, prefix="/api/leases", tags=["Upload"])
```

```
app.include_router(chat.router, tags=["Chat"])

@app.get("/")
async def root() → dict:
    """A simple default route just to show the server is alive."""
    return {"message": "Welcome to the LeaseGPT API!"}
```

> 💡 **NOTE**
> **What just happened?** We created the foundation of our web server. The `lifespan` function is the production-grade way to handle startup and shutdown. On startup, it connects to PostgreSQL and creates any missing tables (so you never have to run a separate setup script). On shutdown, it closes every database connection cleanly via `engine.dispose()`. The `yield` keyword is where the app actually runs — everything before it is startup, everything after is teardown. We also added CORS middleware so our future React frontend won't get blocked by the browser.

2. `backend/app/api/dependencies.py` — Dependency Injection

Here we define the things our routes will need, primarily database and Redis connections.

```
# backend/app/api/dependencies.py

from typing import AsyncGenerator
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import AsyncSessionLocal
from app.config import settings
import redis.asyncio as redis

async def get_db() → AsyncGenerator[AsyncSession, None]:
    """
    Dependency to get a database session.

    When FastAPI calls a route that uses Depends(get_db), it runs this
    function first. The 'async with' block opens a connection from the
    pool and hands it to the route via 'yield'.

    When the route finishes (whether it succeeded or raised an error),
    Python comes back here and the 'async with' block closes the session
    automatically — preventing connection leaks.
    """
    async with AsyncSessionLocal() as session:
        yield session

async def get_redis() → AsyncGenerator[redis.Redis, None]:
    """
    Dependency to get a Redis connection.

    Redis is used for:
    - Caching frequent queries so we don't hammer the database
    - Storing WebSocket session state for the chat feature

    'try/finally' ensures the connection is always closed, even if
    an error occurs inside the route that uses this dependency.
    """
    client = redis.from_url(settings.REDIS_URL, encoding="utf-8", decode_responses=True)
    try:
        yield client
    finally:
        await client.aclose()
```

> ✅ **TIP**
> **Why `yield` instead of `return`?** When you `return` something, the function finishes immediately. When you `yield`, the function pauses, hands the database session to the route, and waits. Once the route finishes its job (whether it succeeded or crashed), the function resumes after the `yield` and safely closes the database connection. It prevents memory leaks!

3. `backend/app/api/middleware.py` — Custom Middleware

Let's add some middleware to track requests and prevent abuse (rate limiting).

```
# backend/app/api/middleware.py

import time
import logging
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import JSONResponse, Response
import redis.asyncio as redis
from app.config import settings

logger = logging.getLogger(__name__)

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    """
    Middleware that logs every incoming request and how long it took to process.
    """
    async def dispatch(self, request: Request, call_next) -> Response:
        start_time = time.time()
        logger.info("Incoming Request: %s %s", request.method, request.url.path)

        response = await call_next(request)

        process_time = time.time() - start_time
        logger.info("Completed Request: %s %s in %.4f secs", request.method, request.url.path,
process_time)
        response.headers["X-Process-Time"] = str(process_time)
        return response

class SimpleRateLimitMiddleware(BaseHTTPMiddleware):
    """
    Sliding-window rate limiter using Redis.
    Allows at most `max_requests` requests per `window_seconds` per client IP.

    Default: 60 requests per 60 seconds (1 request/second average).
    """
    def __init__(self, app, max_requests: int = 60, window_seconds: int = 60):
        super().__init__(app)
        self.max_requests = max_requests
        self.window_seconds = window_seconds
        # Initialize a Redis connection pool for the middleware
        self.redis = redis.from_url(settings.REDIS_URL, encoding="utf-8", decode_responses=True)

    async def dispatch(self, request: Request, call_next) -> Response:
        # Identify the client by their IP address
        client_ip = request.client.host
        now = time.time()
        window_start = now - self.window_seconds
        key = f"rate_limit:{client_ip}"

        # Use a Redis transaction (pipeline) to ensure atomic operations
        async with self.redis.pipeline(transaction=True) as pipe:
            # 1. Remove old timestamps outside the window
            pipe.zremrangebyscore(key, 0, window_start)
```

```
            # 2. Count remaining timestamps in the window
            pipe.zcard(key)
            # 3. Add current timestamp
            pipe.zadd(key, {str(now): now})
            # 4. Set expiry so we don't leak memory for old IPs
            pipe.expire(key, self.window_seconds)

            # Execute all commands at once
            results = await pipe.execute()

        # results[1] is the output of zcard (the number of requests in the window)
        request_count = results[1]

        if request_count ≥ self.max_requests:
            # Client has exceeded the rate limit - return 429 Too Many Requests
            return JSONResponse(
                status_code=429,
                content={"detail": f"Rate limit exceeded. Max {self.max_requests} requests per
{self.window_seconds}s."}
            )

        return await call_next(request)
```

> 💡 **NOTE**
> **What just happened?** The `RequestLoggingMiddleware` acts like a stopwatch. It starts the timer when a request arrives, calls `call_next(request)` to let the rest of the application do its work, and then stops the timer when the response comes back. It logs the time and even adds an `X-Process-Time` header to the final response.

4. `backend/app/api/routes/health.py` — Health Checks

Health checks are crucial for modern deployment systems like Kubernetes or Docker Compose to know if your app is alive.

```
# backend/app/api/routes/health.py

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import text
from app.api.dependencies import get_db

router = APIRouter()

@router.get("/")
async def liveness_check() → dict:
    """
    Liveness probe.
    If this endpoint returns a 200 OK, the web server is running.
    """
    return {"status": "alive"}

@router.get("/ready")
async def readiness_check(session: AsyncSession = Depends(get_db)) → dict:
    """
    Readiness probe.
    Checks if the application is ready to accept traffic by running a
    minimal query against the database. If the query fails, the database
    is not reachable and we return 503 Service Unavailable so that
    Docker / Kubernetes knows not to send traffic here yet.
    """
    try:
        # 'SELECT 1' is the lightest possible query - it returns instantly
```

```
        # and touches no real tables. It just proves the connection works.
        await session.execute(text("SELECT 1"))
        return {"status": "ready", "database": "connected"}
    except Exception as e:
        raise HTTPException(status_code=503, detail=f"Database unavailable: {str(e)}")
```

5. `backend/app/api/routes/upload.py` — File Uploads

Here we handle users uploading their lease PDFs.

```
# backend/app/api/routes/upload.py

import uuid
import logging
from fastapi import APIRouter, File, UploadFile, HTTPException, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from typing import Dict
from app.api.dependencies import get_db
from app.models.lease import LeaseDocument
from app.services.document_processor import process_document
from app.rag.indexer import index_document
from app.config import settings

logger = logging.getLogger(__name__)
router = APIRouter()

# Define allowed file extensions
ALLOWED_EXTENSIONS = {"pdf", "png", "jpg", "jpeg", "tiff"}

@router.post("/")
async def upload_lease(
    file: UploadFile = File(...),  # 'File(...)' tells FastAPI this is a multipart form upload
    db: AsyncSession = Depends(get_db)
) -> Dict[str, str]:
    """
    Endpoint to upload a lease document. Performs the full pipeline:
    1. Validates the file type and size.
    2. Creates a LeaseDocument record in the database.
    3. Extracts text from the PDF using PyMuPDF.
    4. Chunks the text, embeds each chunk, and saves to the database.
    """
    # 1. Validate the file extension
    filename = file.filename or ""
    ext = filename.split(".")[-1].lower() if "." in filename else ""
    if ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(
            status_code=400,
            detail=f"Unsupported file type '{ext}'. Allowed: {ALLOWED_EXTENSIONS}"
        )

    # 2. Read the file into memory
    contents = await file.read()

    # 3. Validate file size
    if len(contents) > settings.MAX_UPLOAD_SIZE_BYTES:
        raise HTTPException(status_code=413, detail=f"File too large. Maximum size is
{settings.MAX_UPLOAD_SIZE_BYTES // (1024 * 1024)}MB.")

    # 4. Create a LeaseDocument record in the database with status 'processing'
    lease = LeaseDocument(
        id=uuid.uuid4(),
        filename=filename,
        status="processing",
        metadata_={"file_size_bytes": len(contents)},
    )
```

```
    db.add(lease)
    await db.commit()
    await db.refresh(lease)  # Reload from DB so we have the generated id

    try:
        # 5. Extract text from the file page by page
        extracted_pages = await process_document(contents, filename)

        # 6. Chunk + embed + save to the database
        chunk_count = await index_document(
            lease_id=str(lease.id),
            extracted_pages=extracted_pages,
            db=db
        )

        # 7. Mark the lease as completed
        lease.status = "completed"
        lease.metadata_ = {**lease.metadata_, "chunk_count": chunk_count}
        await db.commit()

    except Exception as e:
        logger.exception("Processing failed for lease %s", lease.id)
        # If anything goes wrong, mark the lease as failed so the user knows
        lease.status = "failed"
        lease.metadata_ = {**lease.metadata_, "error": str(e)}
        await db.commit()
        raise HTTPException(status_code=500, detail=f"Processing failed: {str(e)}")

    return {
        "lease_id": str(lease.id),
        "message": "File uploaded and processed successfully.",
        "status": "completed",
        "chunks_created": str(chunk_count),
    }
```

> ⚠️ **IMPORTANT**
> **What is a multipart upload?** Normally, HTTP requests send text (like JSON). `multipart/form-data` is a special way to send large binary files (like PDFs or images) over HTTP by breaking them into parts. FastAPI's `UploadFile` handles all the complexity of this for us.

6. Services and Utils (The Business Logic)

Why a Service Layer? Routes should only handle web stuff (receiving requests, validating inputs, returning responses). The actual work (saving to the database, calling S3) should live in "Services". This makes your code reusable.

> 💡 **NOTE**
> In our implementation, we kept the upload pipeline simple by doing all the work directly inside `upload.py` — it calls `process_document()` for text extraction and `index_document()` for chunking and embedding. In a larger production app, you would extract this orchestration into a dedicated `upload_service.py` and use a utility like `utils/s3.py` (using the `boto3` library) to upload files to Amazon S3 instead of processing them in memory. For our single-lease-at-a-time workflow, the inline approach is cleaner and easier to debug.

Chapter 5: Document Processing (Text Extraction)

Now that we have the file uploaded, we need to read the text inside it.

Beginner Concepts

- **What is OCR?** Optical Character Recognition. If you scan a physical piece of paper into a PDF, the computer just sees an image (a bunch of pixels). It doesn't know there are words there. OCR is AI that looks at the pixels and types out the letters it sees.
- **What is AWS Textract?** It's a powerful OCR service provided by Amazon. It's smart enough to understand tables, forms, and signatures.
- **What is PyMuPDF?** It's a fast Python library. If a PDF was created digitally (e.g., exported from Microsoft Word), the text is already embedded inside it. We don't need OCR; PyMuPDF can just rip the text right out instantly.

1. `backend/app/services/document_processor.py`

This service decides how to read the file.

```
# backend/app/services/document_processor.py

import fitz  # This is PyMuPDF (confusing name, I know!)
from typing import List, Dict
import logging

logger = logging.getLogger(__name__)

async def process_document(file_bytes: bytes, filename: str) -> List[Dict]:
    """
    Takes a file, determines if it needs OCR, and extracts the text page by page.
    Returns a list of dicts, one per page: [{"page_num": 1, "text": "..."}, ...]
    """
    extension = filename.split(".")[-1].lower() if "." in filename else ""
    extracted_pages = []

    # ── Image files: need OCR ──────────────────────────────────────────
    # Image files (JPG, PNG, TIFF) have no text layer at all.
    # Production path: send to AWS Textract for OCR.
    # For now we raise a clear error so the user knows to upload a PDF instead.
    if extension in ["jpg", "jpeg", "png", "tiff"]:
        raise NotImplementedError(
            "Image OCR via AWS Textract is not yet configured. "
            "Please upload a digitally-created PDF."
        )

    # ── PDF files ──────────────────────────────────────────────────────
    if extension == "pdf":
        logger.info("PDF detected. Analysing...")
        # Open the PDF entirely from the bytes in memory (no temp file needed)
        pdf_document = fitz.open(stream=file_bytes, filetype="pdf")

        # Heuristic: check the first page for readable text.
        # If it has fewer than 50 characters it is almost certainly a scanned image.
        first_page_text = pdf_document[0].get_text()
        if len(first_page_text.strip()) < 50:
            pdf_document.close()
            raise NotImplementedError(
                "This PDF appears to be scanned (no digital text layer found). "
                "AWS Textract OCR is not yet configured. "
                "Please upload a digitally-created PDF."
```

```
            )

        # Digital PDF — extract text from every page directly
        logger.info("Digital PDF detected. Extracting text from %d pages ...", len(pdf_document))
        for page_num in range(len(pdf_document)):
            page = pdf_document[page_num]
            extracted_pages.append({
                "page_num": page_num + 1,  # 1-indexed so page numbers match the real document
                "text": page.get_text(),
            })

        pdf_document.close()
        return extracted_pages

    raise ValueError(f"Unsupported file extension: '{extension}'")
```

> ✅ **TIP**
> **Why do we check if it's a scanned PDF?** AWS Textract costs money per page. PyMuPDF is completely free and runs locally. By checking if the PDF already contains digital text, we save time and money!

Chapter 6: The RAG Pipeline (Chunking, Embedding, Retrieval)

This is the brain of LeaseGPT. RAG stands for **Retrieval-Augmented Generation**.

Normally, an LLM (like ChatGPT) only knows what it was trained on months ago. It doesn't know about *your* specific lease. RAG solves this. We **Retrieve** relevant parts of your lease, and give them to the LLM to **Augment** its knowledge, so it can **Generate** an accurate answer.

Beginner Concepts

- **What is Chunking?** Imagine a 100-page lease. If a user asks "What is the pet policy?", we shouldn't feed all 100 pages to the AI. AIs have limits on how much they can read at once (context window), and it's expensive. Instead, we break the lease up into small "chunks" (paragraphs). We then search for the *most relevant* chunks and only feed those to the AI.
- **What is a Vector Embedding?** This is the magic of modern AI search. An embedding model takes text (e.g., "The rent is $2000/month") and turns it into a giant array of numbers, like `[0.12, -0.45, 0.88, ...]`. These numbers represent the *meaning* of the text in mathematical space. Sentences with similar meanings will have similar numbers, even if they use completely different words!
- **Semantic Search vs Keyword Search:**
  - **Keyword:** Searching for exactly the word "rent". Misses "monthly payment".
  - **Semantic:** Comparing those arrays of numbers (vectors). Searching for "how much do I owe" will mathematically match the chunk about "monthly rent" because their meanings are close.
- **Reciprocal Rank Fusion (RRF):** We actually want *both* searches. Keyword search is great for exact names or section numbers. Semantic search is great for concepts. RRF is a fancy math formula that takes the top results from both searches and blends them into one super-accurate ranked list.

1. `backend/app/rag/indexer.py` — Breaking it Down

This file takes the extracted text, chunks it, embeds it, and saves it to PostgreSQL.

```
# backend/app/rag/indexer.py

import uuid
import logging
from typing import List, Dict
from sqlalchemy.ext.asyncio import AsyncSession
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.ollama import OllamaEmbedding
from app.models.lease import LeaseDocument, LeaseChunk
from app.config import settings

logger = logging.getLogger(__name__)

# Initialize the embedding model once at module load so we don't recreate it on every request.
# OllamaEmbedding talks to our local Ollama server to convert text into a list of numbers.
embedding_model = OllamaEmbedding(
    model_name=settings.EMBEDDING_MODEL,   # "nomic-embed-text" from our .env file
    base_url=settings.OLLAMA_BASE_URL,     # "http://localhost:11434"
)

async def index_document(
    lease_id: str,
    extracted_pages: List[Dict],
    db: AsyncSession
) -> int:
    """
    Takes a list of extracted pages, chunks them into smaller pieces,
    generates a vector embedding for each chunk, and saves everything
    to the database.

    Returns the total number of chunks created.
    """
    logger.info("Indexing lease %s ...", lease_id)

    # 1. Initialize the sentence splitter.
    #    chunk_size=512  → Each chunk is at most ~512 tokens (roughly 380 words).
    #    chunk_overlap=50 → The last 50 tokens of chunk N are repeated at the
    #                       start of chunk N+1 so context is never cut in half.
    splitter = SentenceSplitter(chunk_size=512, chunk_overlap=50)

    chunk_models: List[LeaseChunk] = []

    # 2. Loop through every page that was extracted from the PDF.
    for page in extracted_pages:
        page_num = page["page_num"]
        text = page["text"]

        if not text.strip():
            # Skip totally blank pages (some PDFs have cover pages with no text)
            continue

        # 3. Break this page's text into smaller chunks.
        text_chunks = splitter.split_text(text)

        for chunk_text in text_chunks:
            # 4. Ask Ollama to embed this chunk.
            #    This is a network call to your local Ollama server.
            #    The result is a list of 768 floating-point numbers.
            embedding: List[float] = await embedding_model.aget_text_embedding(chunk_text)

            # 5. Build a LeaseChunk SQLAlchemy model instance.
            #    We store both the raw text AND its vector embedding in the database.
            chunk = LeaseChunk(
                id=uuid.uuid4(),
                document_id=uuid.UUID(lease_id),
                text_content=chunk_text,
                embedding=embedding,
```

```
                chunk_metadata={"page_number": page_num}
            )
            chunk_models.append(chunk)

    # 6. Bulk-insert all chunks in a single database round-trip.
    #    add_all() stages all the models, commit() writes them to disk.
    db.add_all(chunk_models)
    await db.commit()

    total = len(chunk_models)
    logger.info("Indexing complete! Saved %d chunks for lease %s.", total, lease_id)
    return total
```

> **WARNING**
> **Why do we need overlap?** If Chunk 1 ends with "The tenant is responsible for paying" and Chunk 2 starts with "the water bill," searching for "water bill responsibility" might fail because the concept was cut in half! Overlap ensures context bleeds between chunks.

2. `backend/app/rag/retriever.py` — Finding the Answers

When a user asks a question, this file searches the database for the most relevant chunks.

```
# backend/app/rag/retriever.py

from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession
from llama_index.embeddings.ollama import OllamaEmbedding
from app.config import settings
from typing import List, Dict

# Reuse the same embedding model instance defined in indexer.py.
# In a larger app you would centralise this in a shared module.
embedding_model = OllamaEmbedding(
    model_name=settings.EMBEDDING_MODEL,
    base_url=settings.OLLAMA_BASE_URL,
)

async def hybrid_search(
    db: AsyncSession,
    lease_id: str,
    query_text: str,
    limit: int = 5
) -> List[Dict]:
    """
    Performs a hybrid search combining:
      - Semantic search  (vector similarity via pgvector)
      - Keyword search   (full-text search via PostgreSQL)

    The two result lists are merged using Reciprocal Rank Fusion (RRF),
    which rewards chunks that score well in BOTH searches.

    Returns a list of the top matching chunks, each with its text and page number.
    """

    # 1. Embed the user's question into a 768-dimensional vector.
    #    This lets us compare its meaning against the stored chunk embeddings.
    query_embedding: List[float] = await embedding_model.aget_text_embedding(query_text)

    # pgvector expects the embedding as a string like '[0.1, 0.2, ...]'
    query_embedding_str = "[" + ",".join(str(x) for x in query_embedding) + "]"

    # 2. Build the hybrid search SQL using Common Table Expressions (CTEs).
```

```
#    CTEs (introduced by the WITH keyword) are named sub-queries that make
#    complex SQL readable by breaking it into labelled steps.
rrf_query = text("""
WITH
-- Step A: Semantic Vector Search
--   The ⟺ operator (provided by pgvector) computes cosine distance.
--   Cosine distance is small when two vectors point in the same direction,
--   meaning the texts have similar meaning.
--   ROW_NUMBER() converts raw distances into ranks (1st, 2nd, 3rd …).
semantic_search AS (
    SELECT
        lc.id,
        lc.text_content,
        (lc.chunk_metadata->>'page_number')::int AS page_number,
        ROW_NUMBER() OVER (ORDER BY lc.embedding ⟺ CAST(:query_embedding AS vector)) AS rank
    FROM lease_chunks lc
    WHERE lc.document_id = CAST(:lease_id AS uuid)
    LIMIT :candidate_limit
),

-- Step B: Keyword Full-Text Search
--   to_tsvector() converts text into a searchable bag-of-words index.
--   plainto_tsquery() parses the user's question into search terms.
--   The @@ operator checks for a match.
--   ts_rank() scores how often and how prominently the words appear.
keyword_search AS (
    SELECT
        lc.id,
        lc.text_content,
        (lc.chunk_metadata->>'page_number')::int AS page_number,
        ROW_NUMBER() OVER (
            ORDER BY ts_rank(
                to_tsvector('english', lc.text_content),
                plainto_tsquery('english', :query_text)
            ) DESC
        ) AS rank
    FROM lease_chunks lc
    WHERE lc.document_id = CAST(:lease_id AS uuid)
      AND to_tsvector('english', lc.text_content) @@ plainto_tsquery('english', :query_text)
    LIMIT :candidate_limit
),

-- Step C: Reciprocal Rank Fusion (RRF)
--   Formula: score = 1 / (60 + rank)
--   The constant 60 dampens the effect of very-high ranks so that a
--   chunk ranked #1 in one list doesn't completely dominate over a chunk
--   ranked #2 in both lists.
--   COALESCE handles chunks that only appeared in one of the two lists.
--   A FULL OUTER JOIN keeps every chunk that appeared in either list.
rrf_scores AS (
    SELECT
        COALESCE(s.id, k.id)                                          AS chunk_id,
        COALESCE(s.text_content, k.text_content)                      AS text_content,
        COALESCE(s.page_number, k.page_number)                        AS page_number,
        COALESCE(1.0 / (60.0 + s.rank), 0.0)
            + COALESCE(1.0 / (60.0 + k.rank), 0.0)                   AS rrf_score
    FROM semantic_search s
    FULL OUTER JOIN keyword_search k ON s.id = k.id
)

SELECT chunk_id, text_content, page_number, rrf_score
FROM rrf_scores
ORDER BY rrf_score DESC
LIMIT :limit;
""")

# 3. Run the query against the live database.
#    We pass all user inputs as named parameters (the :name syntax)
```

```
    #    so SQLAlchemy handles escaping and prevents SQL injection.
    result = await db.execute(
        rrf_query,
        {
            "query_embedding": query_embedding_str,
            "query_text": query_text,
            "lease_id": str(lease_id),
            "candidate_limit": limit * 10,  # Cast a wider net before RRF trims to final limit
            "limit": limit,
        }
    )

    rows = result.fetchall()

    # 4. Convert the raw database rows into clean Python dictionaries.
    return [
        {
            "id": str(row.chunk_id),
            "text": row.text_content,
            "page": row.page_number,
            "score": float(row.rrf_score),
        }
        for row in rows
    ]
```

> 💡 **NOTE**
> **What just happened in that SQL?** 1. We found the top 5 chunks that matched the *meaning* (semantic). 2. We found the top 5 chunks that contained the *exact words* (keyword). 3. We gave every chunk a score based on its rank in those lists (Rank 1 gets a high score, Rank 5 gets a lower score). 4. We added the scores together. If a chunk was #1 in semantic AND #1 in keyword, it wins big! We return those top winners.

3. `backend/app/rag/prompts.py` — Prompt Engineering

Finally, we take those relevant chunks and hand them to the AI to answer the question.

```
# backend/app/rag/prompts.py

from typing import List, Dict
import httpx
from app.config import settings

# ─────────────────────────────────────────────────────────────────────────
# System Prompt
# ─────────────────────────────────────────────────────────────────────────
# A System Prompt tells the AI who it is and what rules it must follow.
# Keeping it strict prevents the model from "hallucinating" (making up facts
# that are not in the lease).

QA_SYSTEM_PROMPT = """\
You are a highly accurate legal assistant specialising in residential and \
commercial leases. Your job is to answer the user's question using ONLY the \
context provided below.

Context blocks (excerpts from the lease) are provided with their page numbers.

RULES:
1. Do not use outside knowledge. If the answer is not in the context, say \
"I cannot find the answer to that in the provided lease document."
2. Always cite your sources using the page number in the context. \
Format citations like this: [Page 4].
3. Be concise but thorough.
```

```
4. If there are conflicting statements in the lease, point them out.

CONTEXT:
{context_string}
"""


def build_context_string(retrieved_chunks: List[Dict]) → str:
    """
    Formats retrieved database chunks into a single readable block of text
    that the LLM can easily parse.

    Each chunk is separated by a divider and labelled with its page number
    so the model knows exactly where in the lease each excerpt came from.
    """
    lines = []
    for chunk in retrieved_chunks:
        lines.append(f"---")
        lines.append(f"[Page {chunk['page']}]")
        lines.append(chunk['text'].strip())
    return "\n".join(lines)


async def ask_ollama(system_prompt: str, user_question: str) → str:
    """
    Sends the filled system prompt and the user's question to the local
    Ollama server and streams back the model's reply as a single string.

    We use httpx (an async-friendly HTTP client) so the rest of the
    FastAPI server can continue handling other requests while we wait
    for Ollama to generate its response.
    """
    payload = {
        "model": settings.LLM_MODEL,   # e.g. "llama3.1:8b" from .env
        "stream": False,               # False → wait for the full reply before returning
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user",   "content": user_question},
        ],
    }

    async with httpx.AsyncClient(timeout=120.0) as client:
        response = await client.post(
            f"{settings.OLLAMA_BASE_URL}/api/chat",
            json=payload,
        )
        response.raise_for_status()  # Raise an error if Ollama returned a non-200 status
        data = response.json()

    # Ollama wraps the answer inside data["message"]["content"]
    return data["message"]["content"]


async def answer_question(
    user_question: str,
    retrieved_chunks: List[Dict]
) → str:
    """
    High-level function that ties everything together:
    1. Formats the retrieved chunks into a context string.
    2. Fills the system prompt with that context.
    3. Sends the prompt + question to Ollama.
    4. Returns the model's answer.

    This is the function called by the chat route in Part 3.
    """
    context_string  = build_context_string(retrieved_chunks)
    system_prompt   = QA_SYSTEM_PROMPT.format(context_string=context_string)
```

```
    answer          = await ask_ollama(system_prompt, user_question)
    return answer
```

Wrapping Up Part 2

We now have a backend that can accept a file, extract its text (using OCR or directly), chunk that text, save vector embeddings to PostgreSQL, and perform lightning-fast hybrid search to find answers!

In Part 3, we will build the **AI Agent (LangGraph)** and **Real-Time WebSockets** to stream answers dynamically!